

基于 AI 漏洞挖掘的经验分享

汇报人：张心意

2026 年 8 月 18 日

目录

第一章 参赛准备

第二章 解题过程

第三章 评审要点

第四章 经验分享

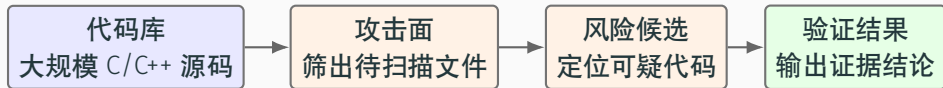
1 参赛准备

准备事项	说明
信息同步	赛题材料和时间节点的调整以官方通知为准，重要变更及时留痕。
环境核验	验证依赖、运行库，区分环境故障与程序问题；注意架构差异（ARM64 / x86_64）。
交付确认	提前核对运行入口（ <i>INSTRUCTION.md</i> ）、日志、交互记录和结果材料。
黑盒纪律	交互记录不含项目名/版本号/CVE 编号；所有测试代码由 AI 生成，禁止人工编写。

2.1 题目名称：基于 AI 的漏洞挖掘

在大模型时代，如何利用大模型的能力，快速、精准地发现存量系统中的 Bug，已成为业界亟待探索的新课题。

验证工程：<https://github.com/tensorflow/tensorflow> (Tags: v2.11.0)



Bug 是什么

定义：指大规模 C/C++ 源码中 **“输入验证缺失”** 族漏洞——用户可控值（张量维度、长度、指针）进入危险运算前缺少守卫。

危害：攻击者向线上 AI 推理 API 注入畸变输入，触发 *SIGFPE/SEGV*，令整个推理容器或服务器硬崩溃——即**拒绝服务**。

2.2 非智能体主流程：精确覆盖与固定验证

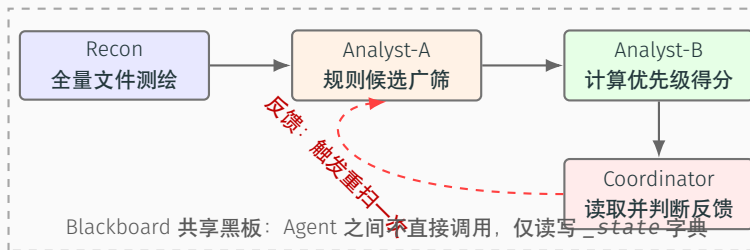
核心阶段	逻辑	输出结果
Recon 攻击面筛选	应用内核、风险、算子三层正则（AND 关系）逐文件严格匹配。	第一层得 634 文件，最终收敛至 98 个 .cc 扫描文件。
SAST 静态扫描	基于 tree-sitter AST 与污染分析；在危险值受控且无守卫时生成候选。	产生 1592 条候选；仅代表代码可疑，不表示漏洞已确认。
EXPECTED 盲测抽检	引入 6 个指定目标为基准，检查 1592 条候选中是否精准包含这些目标。	证明目标未参与前期匹配，真实覆盖率为 6/6，排除硬编码。
PoC 动态验证链	提取指定目标的预置脚本，执行 CONTROL（对照）与 TRIGGER（触发）。	划定分工边界：SAST 负责源码命中，PoC 负责输入触达。
Validator 机械判定	顺序判定：原生信号（PROVEN）→ 目标命中（代码确认）→ 无崩溃（NEGATIVE）。	因运行库含历史修复被拦截，得出 6 条代码级确认，PROVEN=0。

主流程逻辑闭环：当前主入口的本质是验证已知目标的测试基准：EXPECTED 不参与匹配，证明了扫描器的真实覆盖能力；其余自动化利用交由 `auto_exploiter.py` 独立探索。

2.2 非智能体主流程：关键概念

1. **tree-sitter**——增量解析器框架，将源码解析为可遍历语法树；支持增量更新与容错解析，适合大规模 C/C++ 工程。
2. **AST（抽象语法树）**——源码经解析后的树形结构，每个节点对应一个语法构造（函数调用、表达式、声明），是静态分析的基础数据。
3. **污染分析（Taint Analysis）**——追踪用户可控数据（source）到危险运算（sink）的传播路径；若到达 sink 前无守卫（sanitizer），则标记为可疑候选。
4. **PoC（Proof of Concept）**——概念验证脚本，通过构造特定输入触发可疑路径，证明漏洞可在真实运行环境中被触达。
5. **CONTROL / TRIGGER 双轨**——*CONTROL* 为合法对照值（预期不崩溃），*TRIGGER* 为畸变攻击值（预期触发崩溃）；两者对照可排除环境噪声。
6. **PROVEN / 代码级确认 / NEGATIVE**——*PROVEN* 为最高判定级别（PoC 触发原生崩溃信号 *SIGFPE/SEGV*）；代码级确认为 SAST 命中 + PoC 触达但运行库已修复；*NEGATIVE* 为无崩溃产出。

2.3 方案演进：多智能体架构与反馈闭环



前向流水（实线）： Recon 测绘 → Analyst-A 候选广筛 → Analyst-B 优先级打分 → Coordinator 汇总反馈。

反馈闭环（虚线红）： Coordinator 提取下游模式，触发 Analyst-A 重扫，打破单向流水线局限。

阶段总结： 非智能体主流程继续负责固定目标的精确扫描与严格定级。多智能体则专攻“广泛测绘与优先级排序”；它最终输出的是排好序的待验证假设 (*hypotheses*)，不是漏洞证明。

2.3 方案演进：核心机制

核心组件 / 机制	多智能体任务	解决了主流程的什么痛点？
架构设计参考	参考 GitHub 开源项目 <i>usestrix/strix</i> 多智能体安全分析框架。	借鉴成熟 Agent 分工与共享状态思想，避免闭门造车。
引入三类 Agent	<i>Recon</i> 测绘, <i>Analyst-A</i> 找可疑行, <i>Analyst-B</i> 结合“可达性/守卫/差分”打分。	解决 SAST 输出 1592 条候选, 人工无法判断“优先复核哪个”。
Blackboard 黑板	Agent 间不直接对话, 只向 <i>_state</i> 字典贴结果或取线索。	极强解耦; 中间结果全透明留存, 高度可观测。
加入反馈闭环	<i>Analyst-B</i> 筛出高分假设后, 提取模式写入 <i>learned_patterns</i> 。	打破单向流水线, 下游喂回上游, 触发 <i>Analyst-A</i> 重跑。
限制 LLM 范围	仅在 <i>Analyst-B</i> 判断“局部代码有无守卫”时作可选语义辅助介入。	避免滥用大模型; 本地 <i>ReAct</i> 循环无需 API Key 也回退启发式。

Analyst-B 三维打分

可达性——用户可控输入能否沿数据流真正到达危险 sink; **不可达** → **降权**。

守卫——路径上是否存在边界检查 / 输入验证 / 断言等 sanitizer; **有守卫** → **降权**。

差分——候选 API 在**对照输入**与**触发输入**下的行为差异比对; 路径分叉或一处崩溃另一处不崩溃 → 判定**输入敏感**, 优先级提升。三维加权后排序, 最终输出 *hypotheses* 列表: 分数越高越值得优先复核。

2.4 多智能体对本赛题的价值

定位：多智能体做两件事——对 1592 候选做优先级排序（让 PoC 撰写有重点）+ 差分挖掘独立发现新漏洞；PoC 验证由独立流程 `run_all.py` 执行，不归多智能体。

多智能体的两个作用

作用	功能	产出
优先级排序	<i>Analyst-B</i> 读取 1592 SAST 候选 → 聚焦 200 条 → 三维打分（守卫 / 可达性 / 差分）→ <i>confidence</i> ≥ 0.6 才入假设。	<i>hypotheses.json</i> （按置信度降序），让 PoC 撰写有重点，而非 1592 条盲目全做。
差分新发现	看一组同类函数（如 <code>core</code> 与 <code>TFLite</code> 的同名算子）：大多数都做了某项校验，但有少数没做 → 漏的就是 bug。不用 CWE 规则。例： <code>core</code> <code>roll_op.cc</code> 防了零维（ <code>std::max<int>(dim_size, 1)</code> ）， <code>TFLite</code> <code>roll.cc</code> 没防 → 零维 % 0 崩溃。	2 条新漏洞（ <code>roll.cc:218</code> / <code>lsh_projection.cc:119</code> ），SAST 规则未覆盖。

CWE 规则：按照已知的漏洞类型（CWE）预先定义检测规则，用规则匹配代码中的潜在漏洞。

3.1 评审准则

01 准确率与稳定性

所有测试样本的准确率，流程是否能够重复执行。

02 Token 效率

项目运行的 token 总消耗量。

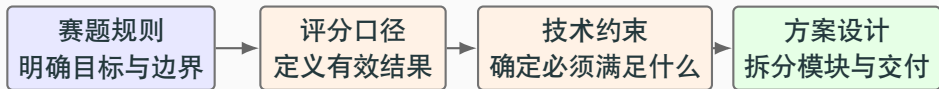
03 人工复核

是否存在 skill 中特定针对测试案例的硬编码或材料缺失。

3.2 竞赛题与难题：评分要点

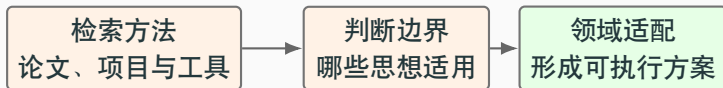
题型	评价维度	关注点
竞赛题	准确率/稳定率、Token 消耗、	高准确率乃至满分较常见；名额有限时，需继续比较资源消耗和稳定性
难题	准确率/稳定率、Token 消耗、人工复核	人工重点检查 skill 中是否存在人工编写针对测试案例的代码，会扣分

4.1 经验一：把规则转化为技术约束



规则要求	转化后的技术约束	本项目中的设计选择
黑盒纪律	不依赖已知漏洞的文件名	使用通用 CWE 风险规则和路径筛选，不把目标答案写入扫描逻辑
结果可复现 证据有边界	输入、中间状态和日志等能够对应 候选、代码级确认和 PROVEN 分开	保存 Blackboard 快照、执行轨迹 将发现链与验证链分离，并按证据强度输出结论

4.2 经验二：检索相关方法，再完成领域适配



步骤	通用做法	本项目中的实践
检索方法	搜索相关论文、开源项目、成熟工具及公开案例	比如 <i>strix</i> AI 自动化渗透与安全测试项目
判断边界	对比应用场景与前提，只保留真正适用的架构思想	借鉴 Agent 分工、共享状态和反馈，不照搬渗透流程
领域适配	把角色、数据流和反馈条件映射到当前任务，并小规模验证	落地 Recon、Analyst-A、Analyst-B、Blackboard 和一次重扫

感谢聆听